



**Waiting for everyone
to join**

**TU257
Information Systems
Marisa Llorens**

Information Systems

Week 8 Introduction to SQL

TU257

Agenda

- Differences between SQLite and MySQL
- Introduction to SQL

Differences between SQLite and MySQL

- SQLite is an open source project available in the public domain
- MySQL is an open source project which is owned by Oracle

Differences between SQLite and MySQL

- SQLite is a **server-less database** and is self-contained. This is also referred to as an embedded database which means the DB engine runs as a part of the app.
- On the other hand, MySQL **requires a server to run**. MySQL will require a client and server architecture to interact over a network.

Differences between SQLite and MySQL

Data Type Support – SQLite vs MySQL

- SQLite supports these datatypes: Blob, Integer, Null, Text, Real.
- MySQL supports the below-mentioned datatypes:
- Tinyint, Smallint, Mediumint, Int, Bigint, Double, Float, Real, Decimal, Double precision, Numeric, Timestamp, Date, Datetime, Char, Varchar, Year, Tinytext, Tinyblob, Blob, Text, MediumBlob, MediumText, Enum, Set, Longblob, Longtext.

Differences between SQLite and MySQL

- **Storage and Portability – SQLite vs MySQL**
- The SQLite library is about 250 KB in size, while the MySQL server is about 600 MB. The SQLite directly stores info in a single file, making it easy to copy. No configurations are required, and the process can be done using minimal support.
- Before copying or exporting MySQL you need to condense it into a single file. For larger databases, this will be a time-consuming activity.

Differences between SQLite and MySQL

Multiple Access – SQLite vs MySQL

- SQLite does not have any specific user management functionality and hence is not suitable for multiple user access.
- MySQL has a well-constructed user management system which can handle multiple users and grant various levels of permission.

Differences between SQLite and MySQL

Scalability – SQLite vs MySQL

- SQLite is suitable for smaller databases. As the database grows the memory requirement also gets larger while using SQLite. Performance optimization is harder when using SQLite.
- On the contrary, MySQL is easily scalable and can handle a bigger database with less effort.

Differences between SQLite and MySQL

Security and Ease of Setup – SQLite vs MySQL

- SQLite does not have an inbuilt authentication mechanism. The database files can be accessed by anyone. However, MySQL comes with a lot of inbuilt security features.
- SQLite does not require too many configurations and is easy to set up. MySQL requires more configurations as compared to SQLite. MySQL also has more setup guides available.

When to Use SQLite

- There are certain times when SQLite can be more effective than the alternative. Some of these scenarios are:
 - Developing small standalone apps
 - Smaller projects which do not require much scalability
 - When you have a requirement to read and write directly from the disk
 - Basic development and testing



[Adobe](#) uses SQLite as the [application file format](#) for their [Photoshop Lightroom](#) product. SQLite is also a standard part of the [Adobe Integrated Runtime \(AIR\)](#). It is reported that [Acrobat Reader](#) also uses SQLite.



[Airbus](#) confirms that SQLite is being used in the flight software for the [A350 XWB](#) family of aircraft.



[Apple](#) uses SQLite in many (most?) of the native applications running on Mac OS-X desktops and servers and on iOS devices such as iPhones and iPods. SQLite is also used in [iTunes](#), even on non-Apple hardware.



[Bentley Systems](#) uses SQLite as the [application file format](#) for their [Microstation](#) CAD/CAM product.



[Bosch](#) uses SQLite in the multimedia systems install on GM, Nissan, and Suzuki automobiles. ([link](#))



The increasingly popular [Dropbox](#) file archiving and synchronization service is reported to use SQLite as the primary data store on the client side.



[Expensify](#) uses SQLite as a server-side database engine for their enterprise-scale expense reporting software.



[Facebook](#) uses SQLite as the SQL database engine in their [osquery](#) product.



[Flame](#) is a malware spy program that is reported to make heavy use of SQLite.



We believe that [General Electric](#) uses SQLite in some product or another because they have written to the SQLite developers at least four separate times requesting the US Export Control Number for SQLite. So presumably GE is using SQLite in something that they are exporting. But nobody (outside of GE) seems to know what that might be.



[Google](#) uses SQLite in their in the [Android](#) cell-phone operating system, and in the [Chrome Web Browser](#).

Well-Known Users of SQLite



When to Use MySQL

- Below are a few scenarios where MySQL is the superior option:
 - Multiple user access to apps
 - When users require strong security and authentication features
 - With distributed systems
 - With apps requiring a larger database
 - With projects which require more scalability
 - ...

SQL

Structured Query Language (SQL)

Introduction

- SQL is a programming language for Relational Databases. It is designed over relational algebra and tuple relational calculus.
SQL comes as a package with all major distributions of RDBMS.

Structured Query Language (SQL)

Introduction

- SQL is based on the relational model:
 - It has many of the same ideas.
 - Databases that support SQL are often described as relational databases.
 - A major strength of the relational model is that it supports **simple, powerful querying of data**.
- Entity Relationship (ER) designs can be implemented in SQL
Entities, attributes, and relationships can all be expressed in terms of SQL.

Structured Query Language (SQL)

Introduction

- Three main sub-languages:
 - **DDL** - Data Definition Language
 - Those commands used to define a database, including creating, altering and dropping tables and adding constraints e.g. **CREATE**, **ALTER**, **TRUNCATE**, **DROP**
 - **DML** - Data Manipulation Language
 - Those commands used to maintain a database, including updating, inserting, and modifying data e.g. **INSERT**, **UPDATE**, **DELETE**
 - **DQL** - Data Query Language
 - Those commands used to query a database, e.g. **SELECT**

Structured Query Language (SQL)

Introduction

- SQL is Relationally Complete (i.e. It implements all operations of relational algebra).
- SQL implements some operations which do not exist in relational algebra (i.e. counting, finding maximal value, sum and other arithmetical operations).
- Operations in SQL (Active & Passive),
 - Active operations update data in the database, while
 - Relational algebra operations are passive: they just retrieve the information which is there.
- In addition SQL can be used from other languages.

Structured Query Language (SQL)

Introduction

- Whitespace is generally ignored in SQL statements and queries, making it easier to format SQL code for readability.
- SQL statements and case sensitivity. Depends on system.
- SQL statements also include the semicolon (;) statement terminator.
 - Though not required on every platform, it is defined as a standard part of the SQL grammar.
- Processing an SQL query takes several steps:
 - The query is checked for validity; -> The query is converted into internal form;
 - The internal form is optimised; and -> The optimised query is executed on the database.

Structured Query Language (SQL)

Introduction

Basic SQL statements include:

- ➔ **CREATE** - A data structure
- ➔ **INSERT** - One or more rows into a table
- ➔ **DELETE** - One or more rows from a table
- ➔ **UPDATE** - Change the column values in a row
- ➔ **DROP** - A data structure
- ➔ **SELECT** - Read one or more rows from a table

SQL CREATE

- SQL uses the command CREATE to define database schema.
- Creates new databases, tables and views from RDBMS.

- Example:

```
CREATE TABLE STUDENTS (  
    ID    INTEGER NOT NULL,  
    Name  TEXT,  
    score INTEGER,  
    PRIMARY KEY( ID )  
);
```

SQL CREATE Examples

Error - If table already exists and you try to execute the create statement

```
CREATE TABLE articles  
(  
  id_article INT(4) NOT NULL,  
  title VARCHAR(50),  
  author VARCHAR(25),  
  price REAL,  
  Primary KEY(id_article)  
);
```

```
CREATE TABLE IF NOT EXISTS  
movies (  
  movie_id    INTEGER,  
  movie_title TEXT,  
  release_datestamp REAL,  
  PRIMARY KEY(movie_id)  
);
```

SQL ALTER

- ALTER

Modifies the table

```
ALTER TABLE students ADD COLUMN module  
TEXT;
```

INSERT

- Insert new values

INSERT INTO Students (ID, Name, score, module)

VALUES (10,"Marie",56, "Maths");

SQL DROP TRUNCATE DELETE

- **DROP**

Deletes the table from the DB

`DROP TABLE movies;`

- **TRUNCATE**

Deletes the data inside a table, but not the table itself.

`TRUNCATE TABLE movies; (MYSQL)`

`DELETE FROM movies; (SQLITE)`

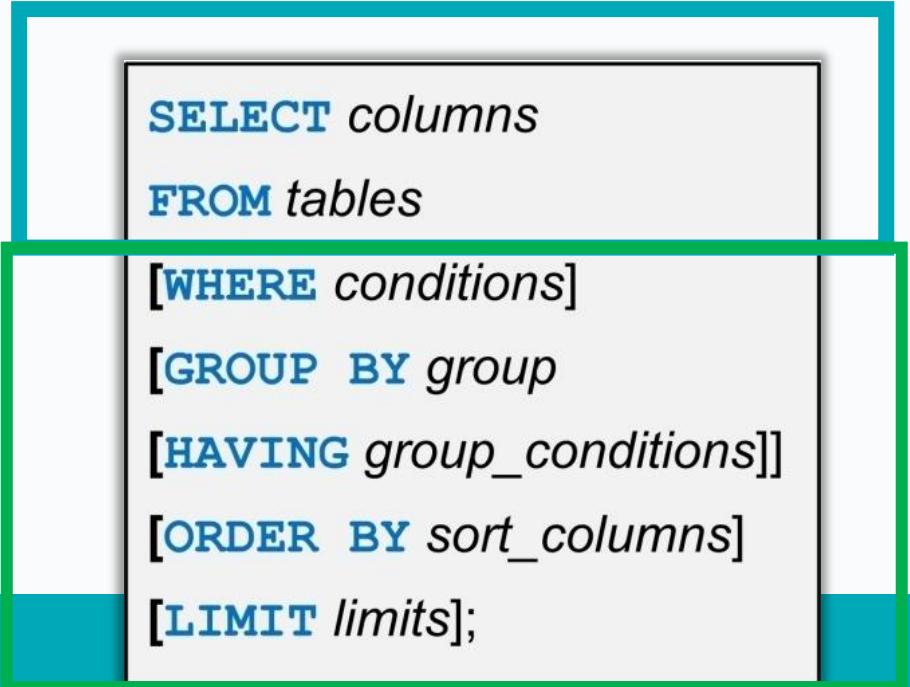
DELETE

- Delete values from table

DELETE FROM students **WHERE** name='Anna';

SELECT/FROM

- The declarative **SELECT** statement can be used to retrieve data from one or more tables.
- The **SELECT** clause is used to retrieve a particular column(s).
- The **FROM** clause indicates the table(s) from which data is to be retrieved.



The diagram illustrates the structure of an SQL SELECT statement. It consists of two main parts: a required part and an optional part. The required part is enclosed in a light blue box and contains the keywords **SELECT** followed by *columns* and **FROM** followed by *tables*. The optional part is enclosed in a light green box and contains several clauses: **[WHERE conditions]**, **[GROUP BY group]**, **[HAVING group_conditions]**, **[ORDER BY sort_columns]**, and **[LIMIT limits];**. The word **Optional** is written in green text to the right of the green box.

```
SELECT columns
```

```
FROM tables
```

```
[WHERE conditions]
```

```
[GROUP BY group
```

```
[HAVING group_conditions]
```

```
[ORDER BY sort_columns
```

```
[LIMIT limits];
```

Optional

SELECT/FROM

- **SELECT *** retrieves all columns

SELECT * FROM students;

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65



ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65

SELECT/FROM

- **SELECT columns** retrieves (a) particular column(s)

SELECT name, score FROM students;

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65



name	score
James	90
Anna	95
Ciara	90
Thomas	65

SELECT/FROM

- The keyword **DISTINCT** removes duplicates from the result set.

SELECT DISTINCT score FROM students;

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65



score
90
95
65

SELECT/FROM

- **Aliases** are alternative names for tables and columns.
- Specify them as follows:

SELECT *column* **AS** *column_alias*

FROM *table* **AS** *table_alias*

SELECT *score* **AS** *grade* **FROM** *students* **AS** *registered*

grade
90
95
90
65

SELECT/FROM/WHERE

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65

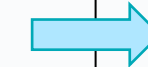
- The **WHERE** clause is used to select rows matching search criteria.

SELECT name, score **FROM** students **WHERE** score > 90;



name	score
Anna	95

SELECT * **FROM** students **WHERE** name = "James";



ID	name	score
1	James	90

SELECT ORDER BY

- The **ORDER BY** clause sorts result rows according to the columns you specify. Default is ascending order, use **DESC** for descending.

SELECT name, score **FROM** students **ORDER BY** name;

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65



name	score
Anna	95
Ciara	90
James	90
Thomas	65

SELECT ORDER BY LIMIT

- The **LIMIT** clause is used to control which rows are returned from the total possible result set.

SELECT name, score **FROM** students **ORDER BY** name **LIMIT** 2;

STUDENTS		
ID	name	score
1	James	90
2	Anna	95
3	Ciara	90
4	Thomas	65



name	score
Anna	95
Ciara	90

SELECT Examples

- **SELECT * FROM** employee;
- **SELECT** name, surname **FROM** employee;
- **SELECT DISTINCT** age **FROM** employee;
- **SELECT** name, surname **FROM** employee **WHERE** age > 60;
- **SELECT** employee_surname **FROM** employee **WHERE** firstname="Mary";
- **SELECT** employee_surname **AS** surname **FROM** employee **WHERE** firstname="Mary";

SELECT Examples

- **SELECT** name, surname **FROM** employee **WHERE** age > 60
ORDER BY surname **LIMIT** 10;
- **SELECT** * **FROM** employee **ORDER BY** surname **DESC**;
- **SELECT** * **FROM** employee **WHERE** Upper(firstname) = 'MARK';

Wildcard Characters

To be used with LIKE

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
^	Represents any character not in the brackets	h[^oa]t finds hit, but not hot and hat
-	Represents a range of characters	c[a-b]t finds cat and cbt

EXAMPLES

- **SELECT * FROM** students **WHERE** name **LIKE** "%s";
- **SELECT** Name **FROM** Students **WHERE** Name **LIKE** "_a%";
- **SELECT** name, score **FROM** Students **WHERE** name **LIKE** "Kier[ao]n";
- **SELECT** location **FROM** office **WHERE** post_code **LIKE** "D08%";
- **SELECT** name, surname **FROM** employee **WHERE** surname **LIKE** "M%";

- Comments start with --

For extra info:

https://www.w3schools.com/sql/sql_select.asp

<https://www.sqlitetutorial.net/sqlite-select/>

SELECT Examples

```
SELECT * FROM students;
```

```
-- SELECT name FROM students;  
-- SELECT name, score FROM students;  
--SELECT DISTINCT score FROM students;  
--SELECT name,score FROM students WHERE score > 90;  
--SELECT * FROM students WHERE name = "James";  
--SELECT * FROM students ORDER BY name;  
--SELECT * FROM students ORDER BY name DESC;  
--SELECT * FROM students ORDER BY score DESC LIMIT 1;  
--SELECT * FROM students WHERE name LIKE "%s";
```

/* more than one line
Comment */

SELECT/FROM/GROUP BY

- The **GROUP BY** clause treats the rows retrieved group by group
- Its main use is in conjunction with group functions like **COUNT()** , **SUM()**, **MAX()**, **MIN()** or **AVG()**

```
SELECT columns  
FROM tables  
[WHERE conditions]  
[GROUP BY group  
[HAVING group_conditions]  
[ORDER BY sort_columns  
[LIMIT limits];
```


SELECT/FROM/GROUP BY

1. Find the average score for each module

```
SELECT AVG(score) FROM students GROUP BY module;
```

2. Find the highest score achieved in the IT module

```
SELECT MAX(score) AS highest_score FROM students WHERE module = 'IT';
```

3. Count the number of students in each module

```
SELECT module, COUNT(*) AS student_count FROM students GROUP BY module;
```

ID	name	score	module
1	James	90	Maths
2	Anna	95	IT
3	Ciara	90	IT
4	Thomas	65	Maths

SELECT/FROM/GROUP BY

- The **HAVING** clause is like a **WHERE** clause for groups.

SELECT module, **AVG**(score) **AS**
average_score **FROM** students **GROUP**
BY module **HAVING** **AVG**(score) > 85;

```
SELECT columns  
FROM tables  
[WHERE conditions]  
[GROUP BY group  
[HAVING group_conditions]]  
[ORDER BY sort_columns]  
[LIMIT limits];
```

SELECT GROUP BY

- **SELECT** module, **AVG**(score) **AS** average_score **FROM** students **GROUP BY** module **HAVING** **AVG**(score) > 85;

ID	name	score	module
1	James	90	Maths
2	Anna	95	IT
3	Ciara	90	IT
4	Thomas	65	Maths

Thanks

- Edmund Nevin for a earlier versions of these lecture notes



**Break
Back at 8pm
for Lab session**

**TU257
Information Systems
Marisa Llorens**